

Assignment 4

Part one

Q1

- result = 8'b00101010 (00101010)
- because the concatenation and replication evaluate as an unsigned and the result is by default unsigned too so the arithmetic shift is useless here

```
module test_shift;

    wire signed [3:0] a = 4'sb1010;
    wire [7:0] result, result2;
    wire [7:0] temp;

    assign temp = {2{a}};
    assign result = {{2{temp[7]}} , temp[7:2]};
    // first approach shifting using concatenation

    assign result2 = $signed({2{a}}) >>> 2;
    // usign the signed conversion

    initial begin
        #0;
        $display("Result: %b", result);
        $display("Result 2: %b", result2);
        $finish;
    end
endmodule
```

Q2

A) Mask Enable Logical: 0001

B) Mask Enable Bitwise: 0000

C) because the reductional operators are only used to check if there are any active or inactive bits in a whole bus and can also be used for the parity check using the xor reductional operator while the logical operators are used to compare two signals and check if one or all of them are non zero signals

Q3)

Evaluation and final values are all answered in comment blocks in the code below:

```
module tb;

    wire [3:0] data = 4'b10X1;
    wire [3:0] ref  = 4'b1001;

    wire [3:0] A = 4'b1010;
    wire [3:0] B = 4'b0101;
    wire [3:0] C = 4'b1100;

    wire flag = ^data == (data === ref) ? 1'b1 : 1'b0;
    /*
        Priority (highest to lowest):
        1. ()
        2. ==
        3. ^
    */

    // ^10X1 == (10X1 === 1001)
    // X == 0
    // flag = X

    wire [3:0] Y = A + B << 1 > C ? A ^ B : A | B;
    // both operations will produce 1111 but i'll do it anyways

    /*
        Priority (highest to lowest):
        1. +
        2. <<
        3. >
    */

    // 1010 + 0101 << 1 > 1100
    // 1111 << 1 > 1100
    // 1110 > 1100
    // Y = 1? 1010^0101 : 1010 " 0101;
    // Y = 1111

    wire [7:0] Result = { {3{^C}}, |B, ~&A, A[2:1] };

    // Result = { {3{^1100}}, |0101, ~&1010, 01 }
    // Result = { {3{0}, 1 ,1 , 01} = 0001101 -> 7 bit -> will expand
```

```
// Result = 00001101

initial begin
    $display(
        "flag = %b, Y = %b, Result = %b",
        flag,
        Y,
        Result
    );
end

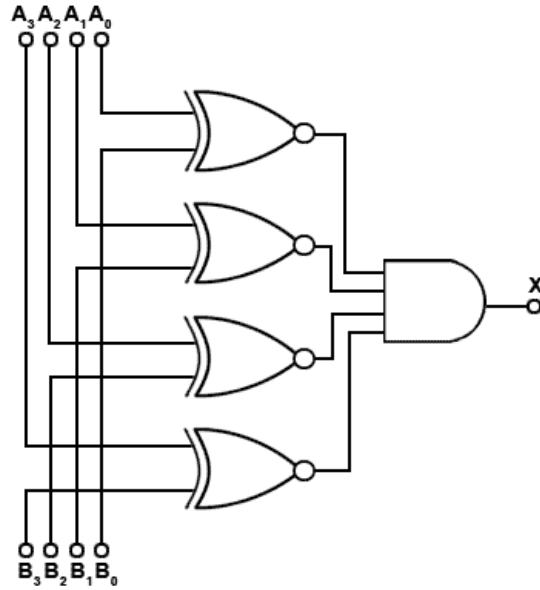
endmodule
```

C) (==) identity operator is used to treat the x and z as a normal values and actually compare them to the corresponding bit in the other variable and that's exactly why it can't be synthesized, because the X and Z aren't real values that you can compare and synthesize it into a real hardware that treats X and Z as a real values

the synthesis tool would synthesize the (data == ref) into a normal operator that compares the two values and output a one or a zero whether they are equal or not

```
data[0] == ref[0]
data[1] == ref[1]
data[2] == ref[2]
data[3] == ref[3]
```

- Each comparator can be done using a single XNOR between every two bits
- Then anding the 4 results together would give you whether the data and ref are equal or not



Q4)

A)

```
out_ternary = X
out_if = 0
```

B) if we changed both inputs into 1 both ternary and if outputs will be one since it is the only value available

Part two

```
module MultiFuncALU (
    input [7:0] A,
    input [7:0] B,
    input Cin,
    input [4:0] Control,
    output reg [15:0] Out
);

    reg [15:0] temp;

    always @(*) begin

        Out = 16'd0;
```

```

temp = 16'd0;

case (Control)

    5'b000000: Out = A + B;
    5'b000001: Out = A + B + Cin;
    5'b000010: Out = A - B + Cin;
    5'b000011: Out = A - B;
    5'b000100: Out = A + 1;
    5'b000101: Out = A - 1;
    5'b000110: Out = A * B + 1;

    5'b000111: Out = A & B;
    5'b001000: Out = A | B;
    5'b001001: Out = A ^ B;
    5'b001010: Out = ~A;
    5'b001011: Out = ~(A & B);

    5'b001100: Out = A << B;
    5'b001101: Out = A >> B;
    5'b001110: Out = A <<< B;
    5'b001111: Out = $signed(A) >>> B;

    5'b100000: begin
        if (B == 0)
            Out = {8'd0, A};
        else
            Out = {8'd0, ((A >> B) | (A << (8-B)))};
    end

    5'b100001: begin
        if (B == 0)
            Out = {8'd0, A};
        else
            Out = {8'd0, ((A << B) | (A >> (8-B)))};
    end

    5'b100010: Out = (A >= B) ? A : B;

    5'b100011: Out = Cin ? B : A;

    5'b100100: Out = {~B, ~A};

    5'b100101: Out = {
        10'd0,

```

```

        A >= B,
        A > B,
        A == B,
        A <= B,
        A < B,
        A != B
    };
    5'b10110: Out = {
        13'd0,
        ~^B,
        ^A
    };

    default: Out = 16'd0;

endcase
end

endmodule

```

```

module MultiFuncALU_tb;

    reg [7:0] A, B;
    reg Cin;
    reg [4:0] Control;
    wire [15:0] Out;

    MultiFuncALU dut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Control(Control),
        .Out(Out)
    );

    task test_operation;
        input [4:0] ctrl;
        input [7:0] test_A;
        input [7:0] test_B;
        input test_Cin;

        begin
            Control = ctrl;
            A = test_A;

```

```

        B = test_B;
        Cin = test_Cin;

        #10;

        $display(
            "Control=%b | A=%h | B=%h | Cin=%b | Out=%h",
            Control, A, B, Cin, Out
        );
    end
endtask

initial begin

    // Arithmetic Operations
    test_operation(5'b00000, 8'h05, 8'h03, 0); // A + B
    test_operation(5'b00001, 8'h05, 8'h03, 1); // A + B + Cin
    test_operation(5'b00010, 8'h08, 8'h03, 1); // A - B + Cin
    test_operation(5'b00011, 8'h08, 8'h03, 0); // A - B
    test_operation(5'b00100, 8'hFF, 8'h00, 0); // A + 1 edge case
    test_operation(5'b00101, 8'h00, 8'h00, 0); // A - 1 edge case
    test_operation(5'b00110, 8'hFF, 8'hFF, 0); // Maximum multiplication

    // Logical Operations
    test_operation(5'b00111, 8'hAA, 8'h55, 0); // AND
    test_operation(5'b01000, 8'hAA, 8'h55, 0); // OR
    test_operation(5'b01001, 8'hAA, 8'h55, 0); // XOR
    test_operation(5'b01010, 8'hAA, 8'h00, 0); // NOT
    test_operation(5'b01011, 8'hAA, 8'h55, 0); // NAND

    // Shift Operations
    test_operation(5'b01100, 8'b00001111, 8'd2, 0); // Logical left
    test_operation(5'b01101, 8'b11110000, 8'd2, 0); // Logical right
    test_operation(5'b01110, 8'b00001111, 8'd2, 0); // Arithmetic left
    test_operation(5'b01111, 8'b11110000, 8'd2, 0); // Arithmetic right

    // Rotate Operations
    test_operation(5'b10000, 8'b10110010, 8'd2, 0); // ROR
    test_operation(5'b10001, 8'b10110010, 8'd2, 0); // ROL

    // Comparison
    test_operation(5'b10010, 8'h10, 8'h20, 0); // MAX
    test_operation(5'b10010, 8'h30, 8'h20, 0); // MAX

```

```

test_operation(5'b10011, 8'hAA, 8'h55, 0); // Cin = 0
test_operation(5'b10011, 8'hAA, 8'h55, 1); // Cin = 1

// Swap / Complement
test_operation(5'b10100, 8'h12, 8'h34, 0);

// Comparison
test_operation(5'b10101, 8'h20, 8'h20, 0); // Equal
test_operation(5'b10101, 8'h30, 8'h20, 0); // A > B
test_operation(5'b10101, 8'h10, 8'h20, 0); // A < B

// Parity
test_operation(5'b10110, 8'hAA, 8'h55, 0);
test_operation(5'b10110, 8'hFF, 8'h00, 0);

// Edge Cases
$display(" EDGE CASES ");

test_operation(5'b00000, 8'hFF, 8'h01, 0);
test_operation(5'b00011, 8'h00, 8'h01, 0);

test_operation(5'b00111, 8'hFF, 8'hFF, 0);
test_operation(5'b01000, 8'h00, 8'h00, 0);
test_operation(5'b01001, 8'hFF, 8'hFF, 0);

test_operation(5'b01100, 8'h01, 8'd7, 0);
test_operation(5'b01101, 8'h80, 8'd7, 0);

// Shift by zero
test_operation(5'b01100, 8'hA5, 8'd0, 0);
test_operation(5'b01101, 8'hA5, 8'd0, 0);

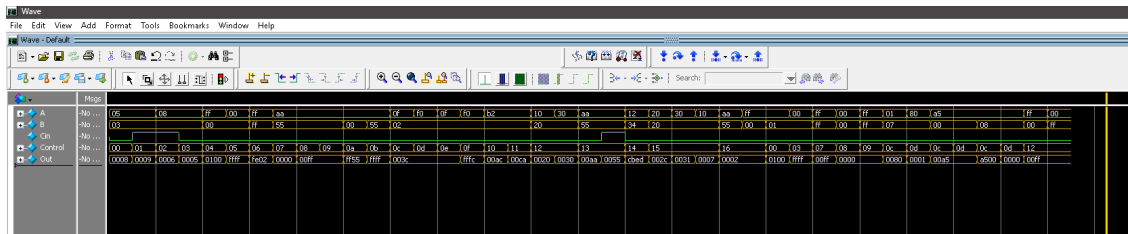
// Shift by maximum amount
test_operation(5'b01100, 8'hA5, 8'd8, 0);
test_operation(5'b01101, 8'hA5, 8'd8, 0);

// Maximum values
test_operation(5'b10010, 8'hFF, 8'h00, 0);
test_operation(5'b10010, 8'h00, 8'hFF, 0);

$finish;
end

```


endmodule



```
sim:/MultiFuncALU_tb/Control \
sim:/MultiFuncALU_tb/Out
/SIM4> run -all
# Control=00000 | A=05 | B=03 | Cin=0 | Out=0008
# Control=00001 | A=05 | B=03 | Cin=1 | Out=0009
# Control=00010 | A=08 | B=03 | Cin=1 | Out=0006
# Control=00011 | A=08 | B=03 | Cin=0 | Out=0005
# Control=00100 | A=ff | B=00 | Cin=0 | Out=0100
# Control=00101 | A=00 | B=00 | Cin=0 | Out=ffff
# Control=00110 | A=ff | B=ff | Cin=0 | Out=fe02
# Control=00111 | A=aa | B=55 | Cin=0 | Out=0000
# Control=01000 | A=aa | B=55 | Cin=0 | Out=00ff
# Control=01001 | A=aa | B=55 | Cin=0 | Out=00ff
# Control=01010 | A=aa | B=00 | Cin=0 | Out=ff55
# Control=01011 | A=aa | B=55 | Cin=0 | Out=ffff
# Control=01100 | A=0f | B=02 | Cin=0 | Out=003c
# Control=01101 | A=f0 | B=02 | Cin=0 | Out=003c
# Control=01110 | A=0f | B=02 | Cin=0 | Out=003c
# Control=01111 | A=f0 | B=02 | Cin=0 | Out=fffc
# Control=10000 | A=b2 | B=02 | Cin=0 | Out=00ac
# Control=10001 | A=b2 | B=02 | Cin=0 | Out=00ca
# Control=10010 | A=10 | B=20 | Cin=0 | Out=0020
# Control=10010 | A=30 | B=20 | Cin=0 | Out=0030
# Control=10011 | A=aa | B=55 | Cin=0 | Out=00aa
# Control=10011 | A=aa | B=55 | Cin=1 | Out=0055
# Control=10100 | A=12 | B=34 | Cin=0 | Out=cbed
# Control=10101 | A=20 | B=20 | Cin=0 | Out=002c
# Control=10101 | A=30 | B=20 | Cin=0 | Out=0031
# Control=10101 | A=10 | B=20 | Cin=0 | Out=0007
# Control=10110 | A=aa | B=55 | Cin=0 | Out=0002
# Control=10110 | A=ff | B=00 | Cin=0 | Out=0002
# EDGE CASES
# Control=00000 | A=ff | B=01 | Cin=0 | Out=0100
# Control=00011 | A=00 | B=01 | Cin=0 | Out=ffff
# Control=00111 | A=ff | B=ff | Cin=0 | Out=00ff
# Control=01000 | A=00 | B=00 | Cin=0 | Out=0000
# Control=01001 | A=ff | B=ff | Cin=0 | Out=0000
# Control=01100 | A=01 | B=07 | Cin=0 | Out=0080
# Control=01101 | A=80 | B=07 | Cin=0 | Out=0001
# Control=01100 | A=a5 | B=00 | Cin=0 | Out=00a5
# Control=01101 | A=a5 | B=00 | Cin=0 | Out=00a5
# Control=01100 | A=a5 | B=08 | Cin=0 | Out=a500
# Control=01101 | A=a5 | B=08 | Cin=0 | Out=0000
# Control=10010 | A=ff | B=00 | Cin=0 | Out=00ff
# Control=10010 | A=00 | B=ff | Cin=0 | Out=00ff
# ** Note: $finish : D:/NTI - Digital IC Design/Session 5/Assignment/multi funciton ALU/MultiFunc_ALU_tb.v(111)
# Time: 410 ns Iteration: 0 Instance: /MultiFuncALU_tb
```

Part three

```
module Data_scrambler (
    input signed [7:0] data_in,
    input [1:0] scramble_key, op_mode,
    input sleep_mode,
    output reg [7:0] data_out,
    output parity_err,
```

```

    output  is_zero
);

assign parity_err = ~^data_out;
assign is_zero = !data_out;

always @(*) begin
    if (sleep_mode)
        data_out = 8'd0;
    else begin
        case (op_mode)
            2'b00: data_out = data_in;
            2'b01: data_out = data_in ^ {4{scramble_key}};
            2'b10: data_out = data_in >>> 2;
            2'b11: data_out = {data_in[3:0], data_in[7:4]};
            default: data_out = 8'd0;
        endcase
    end
end
endmodule

```

```

module Data_scrambler_tb;

    reg signed [7:0] data_in;
    reg [1:0] scramble_key, op_mode;
    reg sleep_mode;
    wire [7:0] data_out;
    wire parity_err, is_zero;
    integer i;

    Data_scrambler dut (
        .data_in(data_in),
        .scramble_key(scramble_key),
        .op_mode(op_mode),
        .sleep_mode(sleep_mode),
        .data_out(data_out),
        .parity_err(parity_err),
        .is_zero(is_zero)
    );

    initial begin
        sleep_mode = 1;

        #10;
    end
endmodule

```

```

sleep_mode = 0;

for (i = 0; i < 4 ; i = i + 1 ) begin
    op_mode = i;
    data_in = 8'hA8; //10101000
    scramble_key = 2'b10;

    #10;

    if (op_mode == 2'b10) begin
        if (data_out != 8'b11101010) begin
            $display("ERROR");
            $stop;
        end
    end

end

$finish;
end

endmodule

```

